

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
GNU nano 7.2 s3t1.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char command[100];

    while (1)
    {
        // Display prompt
        printf("myshell> ");

        // Read user input
        fgets(command, sizeof(command), stdin);

        // Remove newline character
        command[strcspn(command, "\n")] = '\0';

        // Handle exit condition
        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        // Handle empty input
        if (strlen(command) == 0)
        {
            continue;
        }

        // Display entered command
        printf("You entered: %s\n", command);
    }

    return 0;
}
```

```
v_gopinath@DESKTOP-3QLP53Q:~$ nano s3t1.c
v_gopinath@DESKTOP-3QLP53Q:~$ gcc s3t1.c -o s3t1
v_gopinath@DESKTOP-3QLP53Q:~$ ./s3t1
myshell> ls
You entered: ls
myshell> pwd
You entered: pwd
myshell> date
You entered: date
myshell> exit
Exiting shell...
```

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. Open()
2. Read()
3. Write()
4. Close()

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int fd1, fd2;
    char buffer[100];
    int n;

    // Open File1.txt for reading
    fd1 = open("File1.txt", O_RDONLY);

    if (fd1 < 0)
    {
        perror("Error opening File1.txt");
        return 1;
    }

    // Open File2.txt for writing
    // Create it if it does not exist
    // Empty it if it already exists
    fd2 = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    if (fd2 < 0)
    {
        perror("Error opening File2.txt");
        close(fd1);
        return 1;
    }

    // Read from File1.txt and write to File2.txt
    while ((n = read(fd1, buffer, sizeof(buffer))) > 0)
    {
        if (write(fd2, buffer, n) != n)
        {
            perror("Error writing to File2.txt");
            close(fd1);
            close(fd2);
            return 1;
        }
    }

    if (n < 0)
    {
        perror("Error reading File1.txt");
    }

    // Close both files
    close(fd1);
    close(fd2);
}

```

```

v_gopinath@DESKTOP-3QLP53Q:~$ nano s3t2.c
v_gopinath@DESKTOP-3QLP53Q:~$ nano copy.c
v_gopinath@DESKTOP-3QLP53Q:~$ nano file1.txt
v_gopinath@DESKTOP-3QLP53Q:~$ gcc copy.c -o copy
v_gopinath@DESKTOP-3QLP53Q:~$ ./copy

```

Task 3: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```
GNU nano 7.2                                     s3t3.c *
#include <stdio.h>
#include <unistd.h>
#include <string.h>

#define BUFFER_SIZE 100

int main()
{
    char buffer[BUFFER_SIZE];
    int index = 0;
    char ch;

    printf("Simple Command Interface\n");
    printf("Type a command and press Enter.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        index = 0;

        printf("Command: ");
        fflush(stdout);

        while (1)
        {
            // Read one character from keyboard
            if (read(0, &ch, 1) <= 0)
            {
                return 1;
            }

            // Handle Enter key
            if (ch == '\n')
            {
                buffer[index] = '\0';
                printf("\n");

                break;
            }

            // Handle Backspace
            if (ch == 127 || ch == '\b')
            {
                if (index > 0)
                {
                    index--;

                    // Erase character from terminal
                    write(1, "\b \b", 3);
                }

                continue;
            }

            // Store normal characters in buffer
            if (index < BUFFER_SIZE - 1)
            {
                buffer[index] = ch;
                index++;
            }
        }
    }
}
```

```

        // Handle Backspace
        if (ch == 127 || ch == '\b')
        {
            if (index > 0)
            {
                index--;

                // Erase character from terminal
                write(1, "\b \b", 3);
            }

            continue;
        }

        // Store normal characters in buffer
        if (index < BUFFER_SIZE - 1)
        {
            buffer[index] = ch;
            index++;

            // Display typed character
            write(1, &ch, 1);
        }
    }

    // Check for exit command
    if (strcmp(buffer, "exit") == 0)
    {
        printf("Exiting program...\n");
        break;
    }

    // Display the entered command
    printf("You entered: %s\n", buffer);
}

return 0;
}

```

```

v_gopinath@DESKTOP-3QLP53Q:~$ nano s3t3.c
v_gopinath@DESKTOP-3QLP53Q:~$ gcc s3t3.c -o s3t3
v_gopinath@DESKTOP-3QLP53Q:~$ ./s3t3
Simple Command Interface
Type a command and press Enter.
Type 'exit' to quit.

Command: hello
hello
You entered: hello
Command: os
os
You entered: os
Command: exit
exit
Exiting program...
v_gopinath@DESKTOP-3QLP53Q:~$ |

```